

CREDICORE

REPORTE DE MIGRACIÓN E INTELIGENCIA DE DATOS

FASE 2 — INGESTA MASIVA E INTELIGENCIA DE NEGOCIOS

Base de Datos I
Universidad Mariano Gálvez de Guatemala

03 de septiembre de 2026

Estudiante: _____

Ingeniero: _____

1. Introducción

Una base de datos estructurada adquiere valor cuando puede recibir, validar y analizar información proveniente de diferentes fuentes. En esta fase de CrediCore se transformaron datos provenientes de scripts SQL, Excel y un archivo plano para incorporarlos a SQL Server 2022.

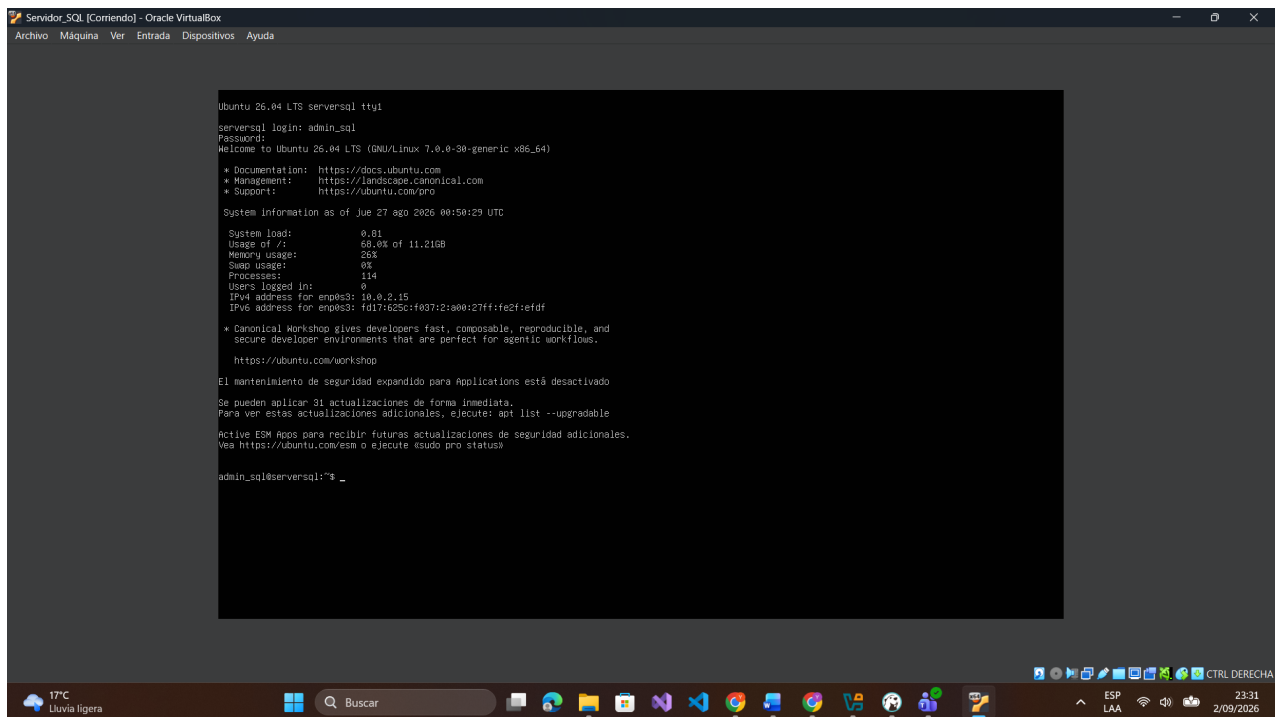
La solución se ejecutó con SQL Server 2022 dentro de Docker sobre Ubuntu 26.04 LTS, utilizando DBeaver. El proceso incluyó generación de datos, manejo de batches, automatización DML, transferencia de archivos, BULK INSERT y consultas de Inteligencia de Negocios.

2. Objetivos

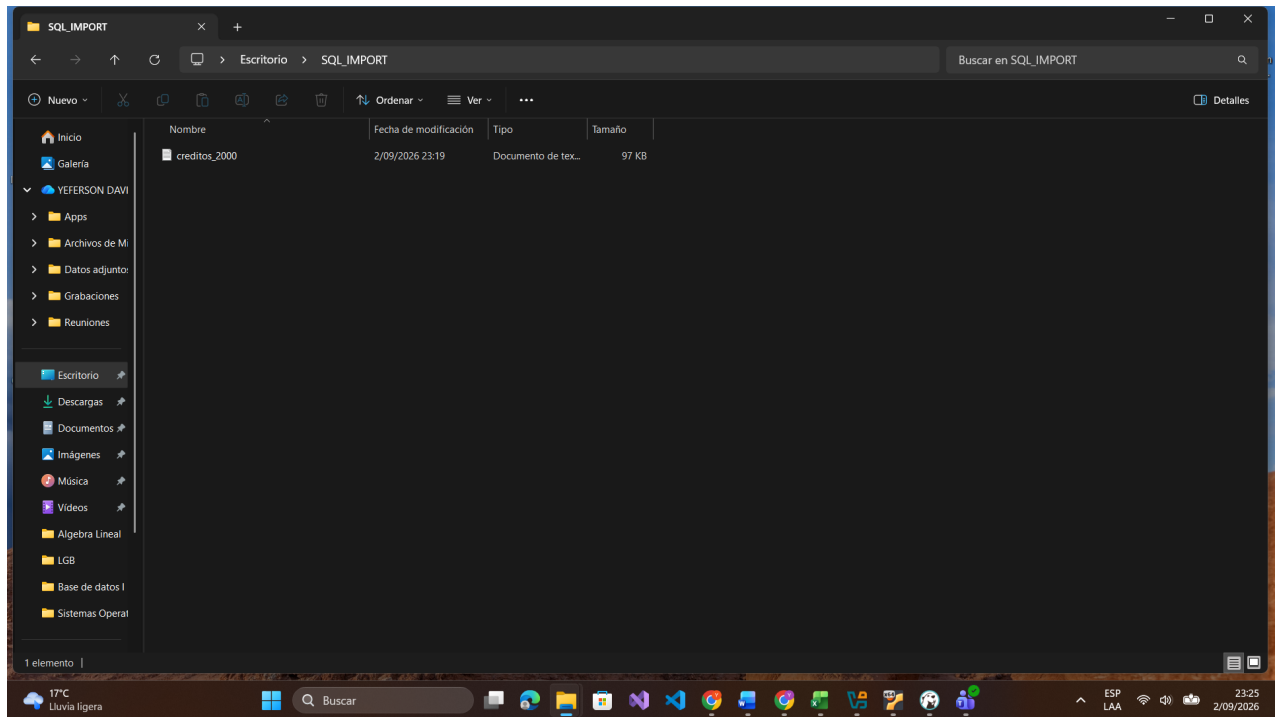
- Generar e insertar 1,500 vehículos y demostrar el límite de 1,000 filas.
- Automatizar 500 INSERT de clientes con Excel y CONCATENAR.
- Cargar 2,000 créditos desde un archivo delimitado por |.
- Resolver el acceso del archivo desde Docker/Ubuntu.
- Aplicar funciones de agregación y HAVING.
- Documentar resultados y tiempos de respuesta.

3. Arquitectura y entorno

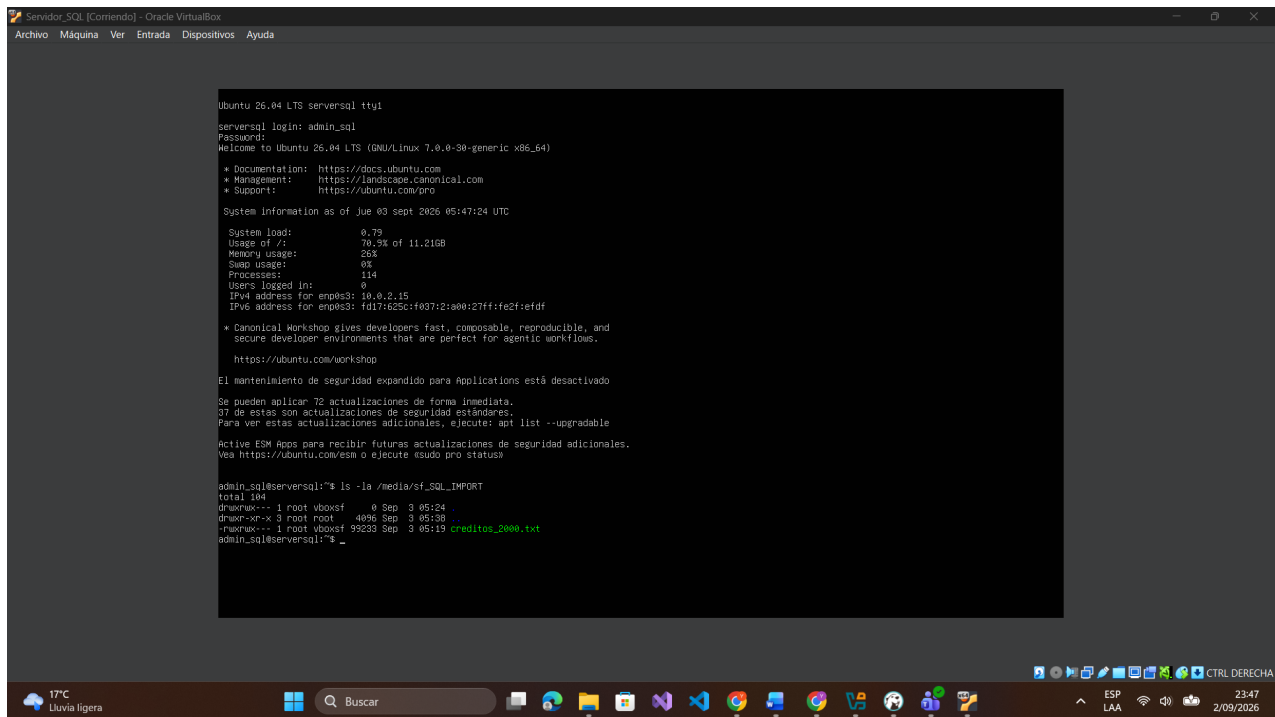
Windows → SQL_IMPORT → Ubuntu 26.04 LTS → Docker/sqlserver2022 → SQL Server 2022 → DBeaver.



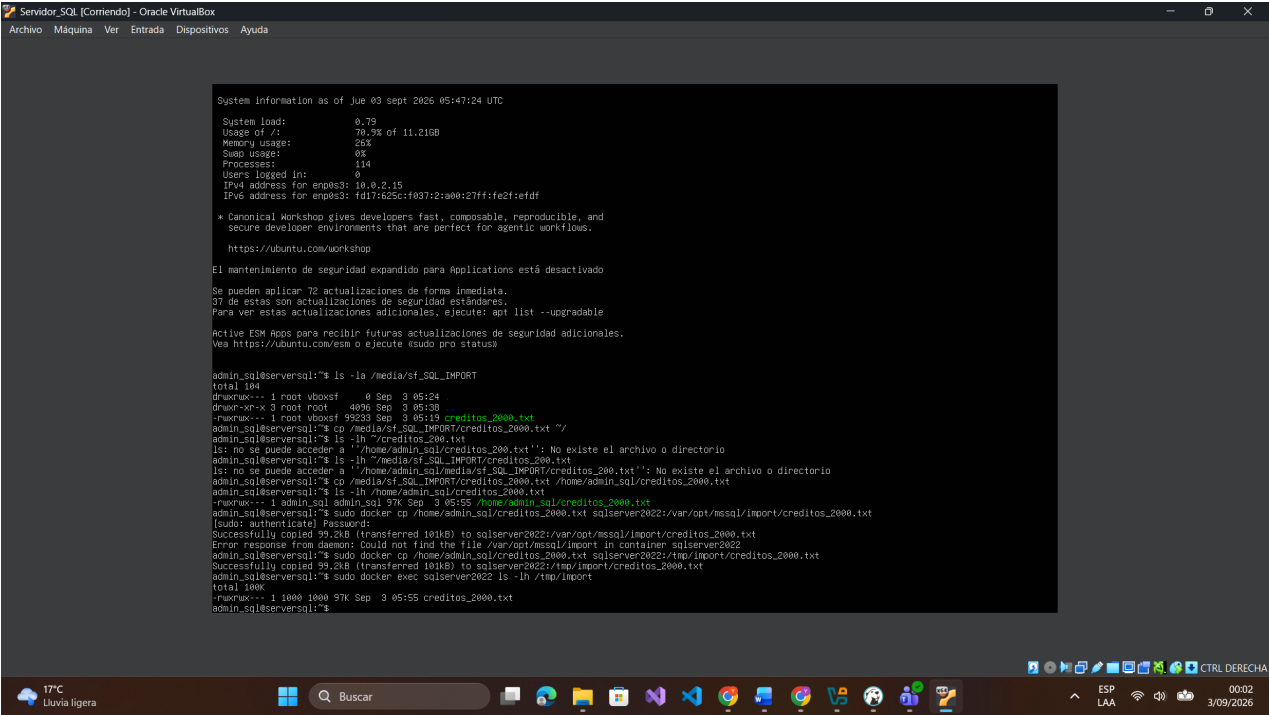
Evidencia 1. Máquina virtual Servidor_SQL con Ubuntu 26.04 LTS.



Evidencia 2. Carpeta SQL_IMPORT en Windows.



Evidencia 3. Ubuntu reconoce /media/sf_SQL_IMPORT y creditos_2000.txt.



Evidencia 4. Transferencia exitosa del archivo al contenedor y verificación en /tmp/import.

4. Base de datos

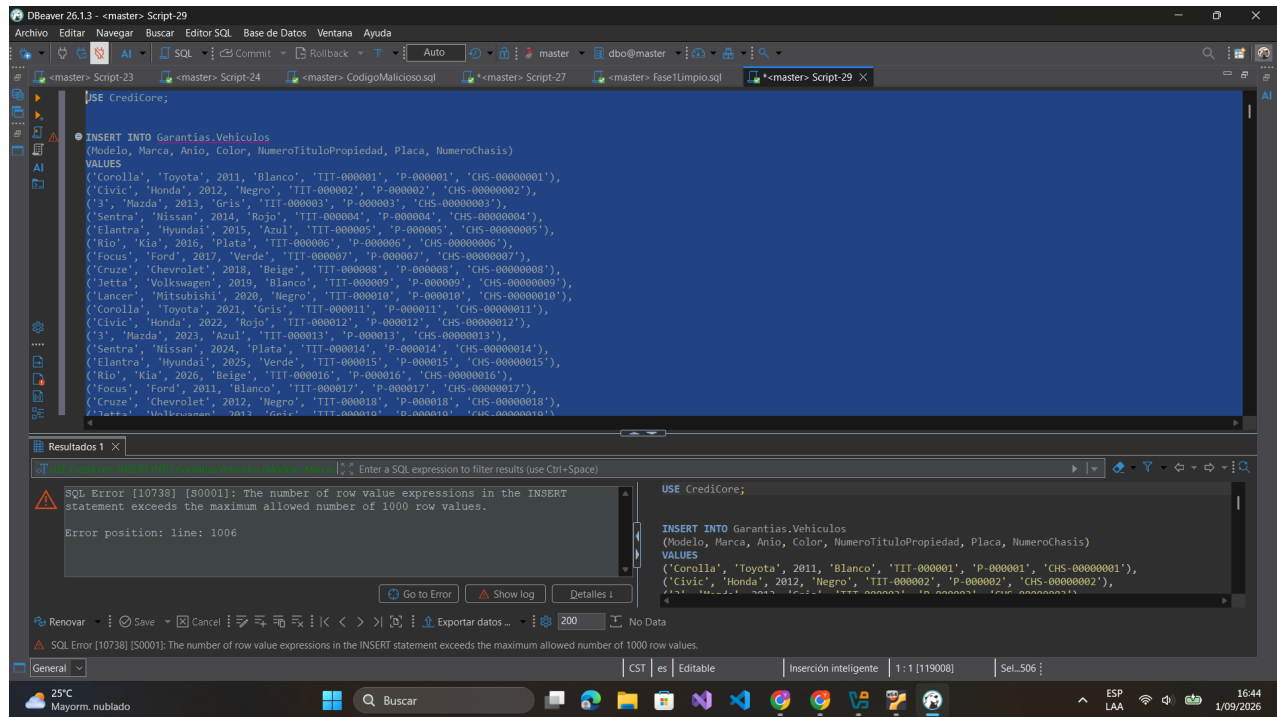
Se reutilizaron Garantias.Vehiculos, Operaciones.Clientes y Operaciones.Creditos. IdCredito es IDENTITY(1,1), por lo que se genera automáticamente y se omitió del INSERT final.

Campo	Tipo	Característica
IdCredito	INT	IDENTITY(1,1), PRIMARY KEY
IdCliente	INT	NOT NULL
IdVehiculo	INT	NOT NULL
MontoCapital	DECIMAL(18,2)	NOT NULL; CHECK > 1000
TasaInteresMensual	DECIMAL(5,2)	NOT NULL; CHECK >= 0
Estado	VARCHAR(20)	NOT NULL; DEFAULT 'activo'
FechaDesembolso	DATETIME	NOT NULL; DEFAULT GETDATE()

5. Parte A.1 — 1,500 vehículos

Se generó un script T-SQL con 1,500 vehículos y se ejecutó como una sola sentencia para provocar el límite solicitado.

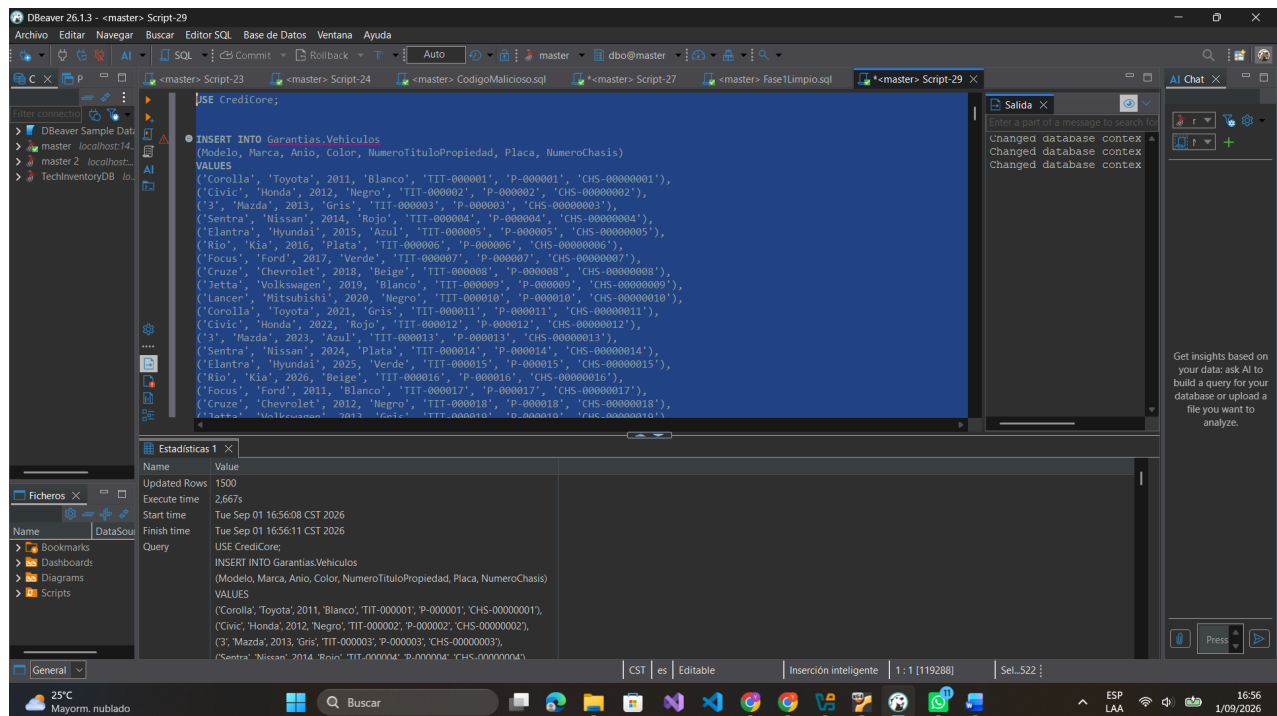
```
USE CrediCore; GO INSERT INTO Garantias.Vehiculos (Modelo, Marca, Anio, Color, NumeroTituloPropiedad, Placa, NumeroChasis) VALUES ('Corolla', 'Toyota', 2011, 'Blanco', 'TIT-000001', 'P-000001', 'CHS-00000001'), ('Civic', 'Honda', 2012, 'Negro', 'TIT-000002', 'P-000002', 'CHS-00000002'), ... -- 1,500 registros generados GO
```



Evidencia 5. Error 10738: el INSERT excede el máximo de 1,000 valores.

La solución fue dividir el script en lotes y utilizar GO para separar los batches.

```
USE Credicore; GO -- Lote 1: registros 1-500 INSERT INTO Garantias.Vehiculos (...) VALUES (...); GO -- Lote 2:
registros 501-1000 INSERT INTO Garantias.Vehiculos (...) VALUES (...); GO -- Lote 3: registros 1001-1500 INSERT
INTO Garantias.Vehiculos (...) VALUES (...); GO
```

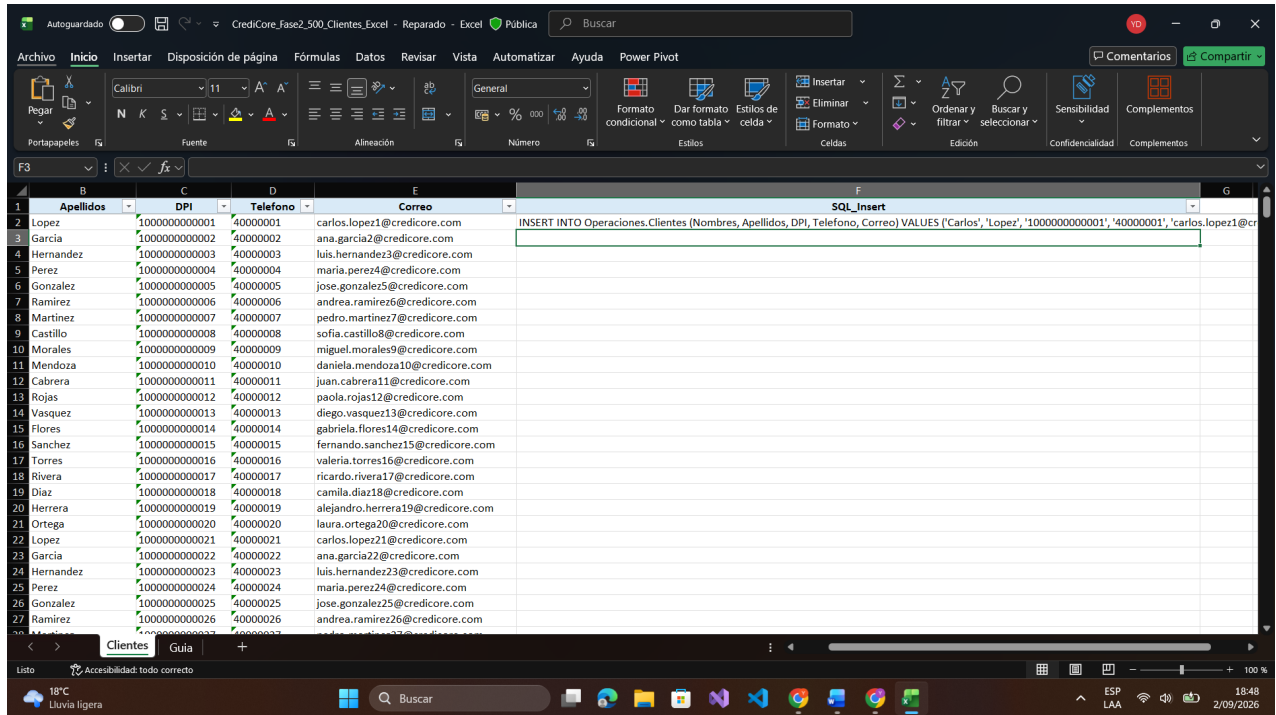


Evidencia 6. Carga corregida: 1,500 filas actualizadas.

6. Parte A.2 — 500 clientes con Excel

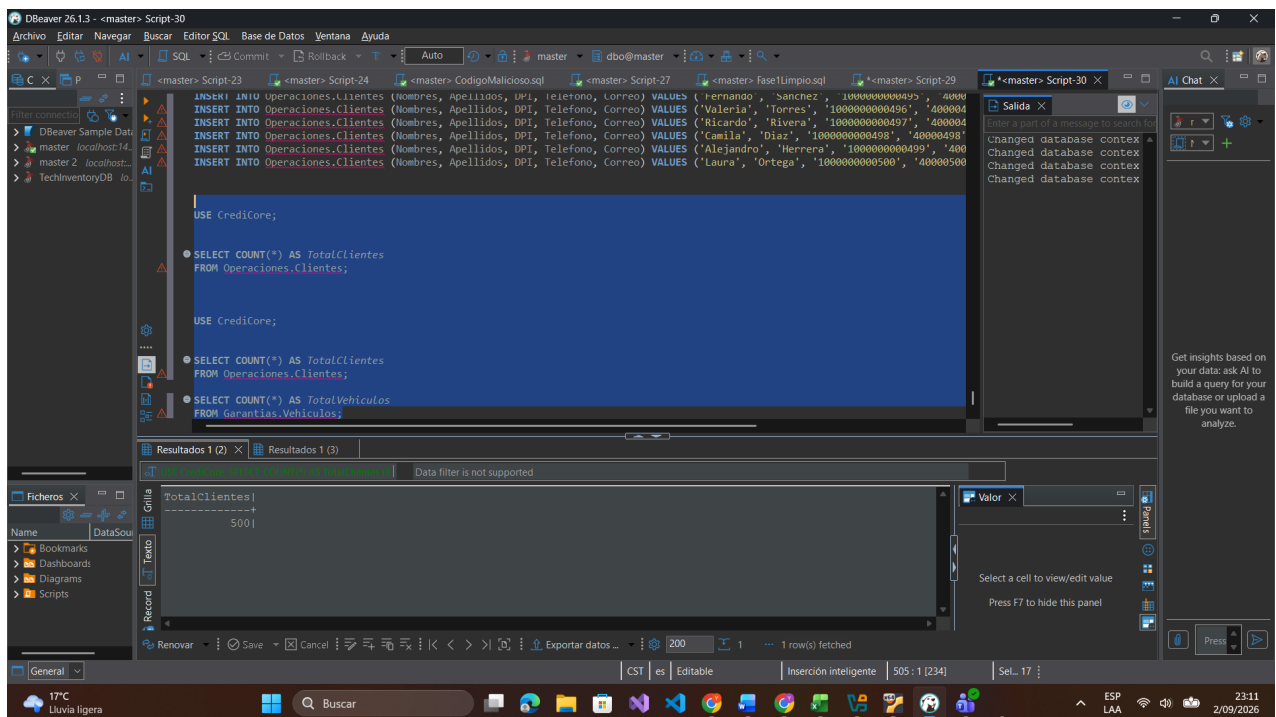
Se utilizó CONCATENAR para construir dinámicamente una sentencia INSERT por cada fila.

```
=CONCATENAR("INSERT INTO Operaciones.Clientes (Nombres, Apellidos, DPI, Telefono, Correo) VALUES ('";A2;"",
";B2;"", ";C2;"", ";D2;"", ";E2;"");")
```



Evidencia 7. Excel con SQL_Insert.

Las sentencias se ejecutaron en DBeaver y se verificó el total de clientes.



Evidencia 8. Verificación: 500 clientes.

7. Parte A.3 — Archivo plano

creditos_2000.txt utiliza | como delimitador y contiene seis campos: referencia, IdCliente, MontoCapital, TasaInteresMensual, Estado y FechaDesembolso.

```

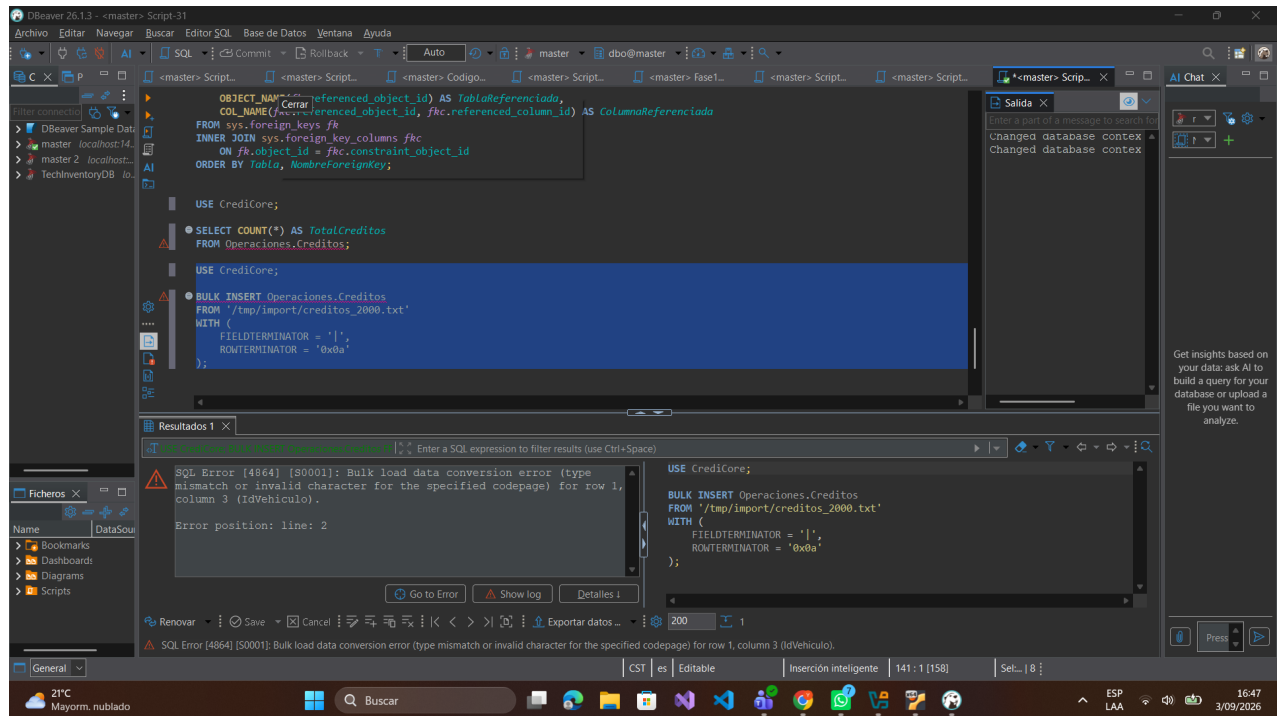
1|1|1637.00|1.57|Activo|2025-01-02 11:00:00 2|2|1774.00|1.64|Pagado|2025-01-03 14:00:00
3|3|1911.00|1.71|Vencido|2025-01-04 17:00:00 4|4|2048.00|1.78|Cancelado|2025-01-05 10:00:00
5|5|2185.00|1.85|Activo|2025-01-06 13:00:00 6|6|2322.00|1.92|Pagado|2025-01-07 16:00:00
7|7|2459.00|1.99|Vencido|2025-01-08 09:00:00 8|8|2596.00|2.06|Cancelado|2025-01-09 12:00:00
9|9|2733.00|2.13|Activo|2025-01-10 15:00:00 10|10|2870.00|2.20|Pagado|2025-01-11 08:00:00

```

La muestra anterior contiene las primeras diez líneas del archivo, requisito que también debe incorporarse al repositorio de GitHub.

8. BULK INSERT — error y solución

El primer intento directo presentó el error 4864 porque la estructura del archivo no coincidía directamente con las columnas de Operaciones.Creditos.



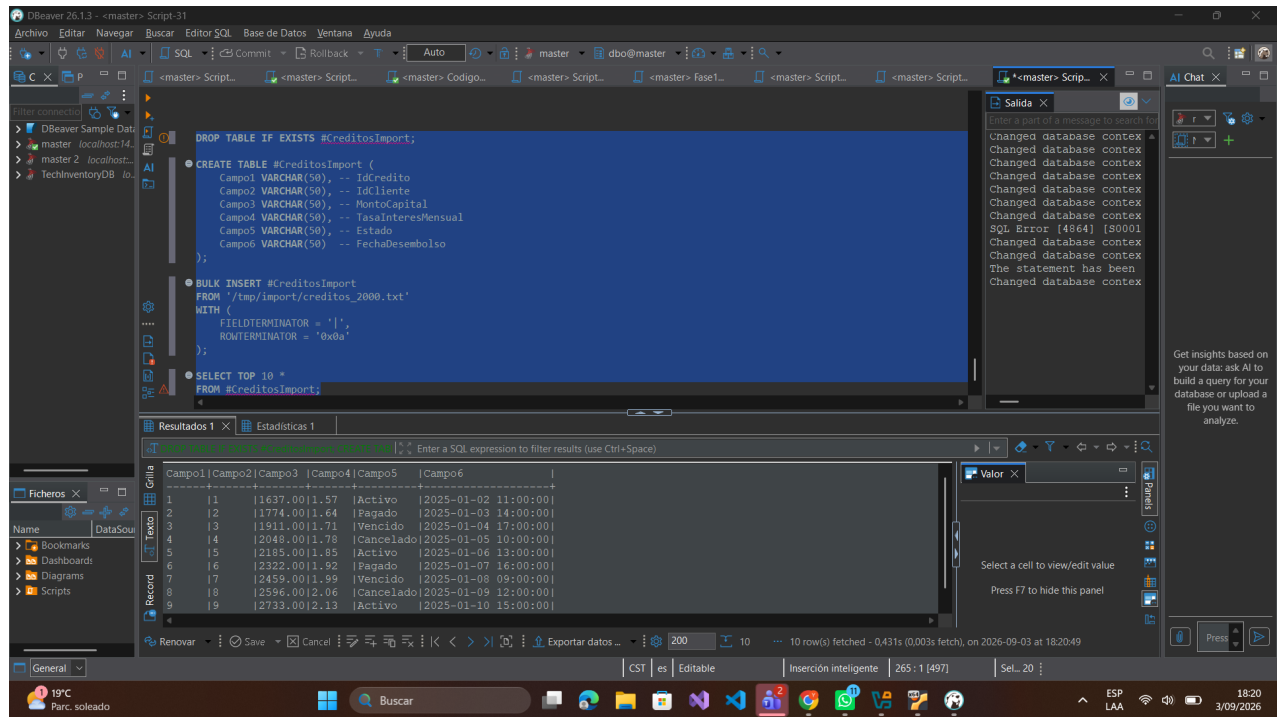
Evidencia 9. Error 4864 durante el primer BULK INSERT.

Se creó una tabla temporal de staging con seis columnas VARCHAR y se volvió a ejecutar BULK INSERT.

```

CREATE TABLE #CreditosImport ( Campo1 VARCHAR(50), Campo2 VARCHAR(50), Campo3 VARCHAR(50), Campo4 VARCHAR(50),
Campo5 VARCHAR(50), Campo6 VARCHAR(50) ); BULK INSERT #CreditosImport FROM '/tmp/import/creditos_2000.txt' WITH
( FIELDTERMINATOR = '|', ROWTERMINATOR = '0x0a' );

```



Evidencia 10. BULK INSERT correcto hacia staging.

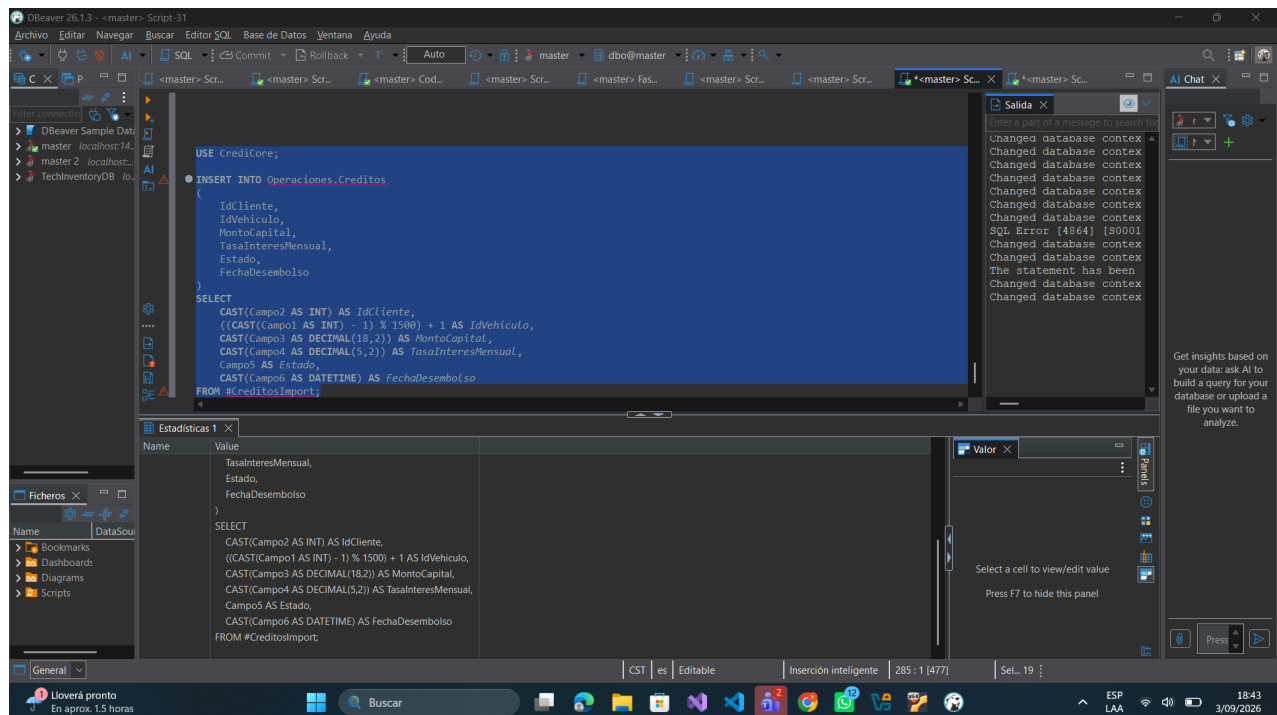
9. Transformación y carga final

Campo2 → IdCliente; Campo3 → MontoCapital; Campo4 → TasaInteresMensual; Campo5 → Estado; Campo6 → FechaDesembolso. IdVehiculo se asignó cíclicamente sobre los 1,500 vehículos disponibles.

```

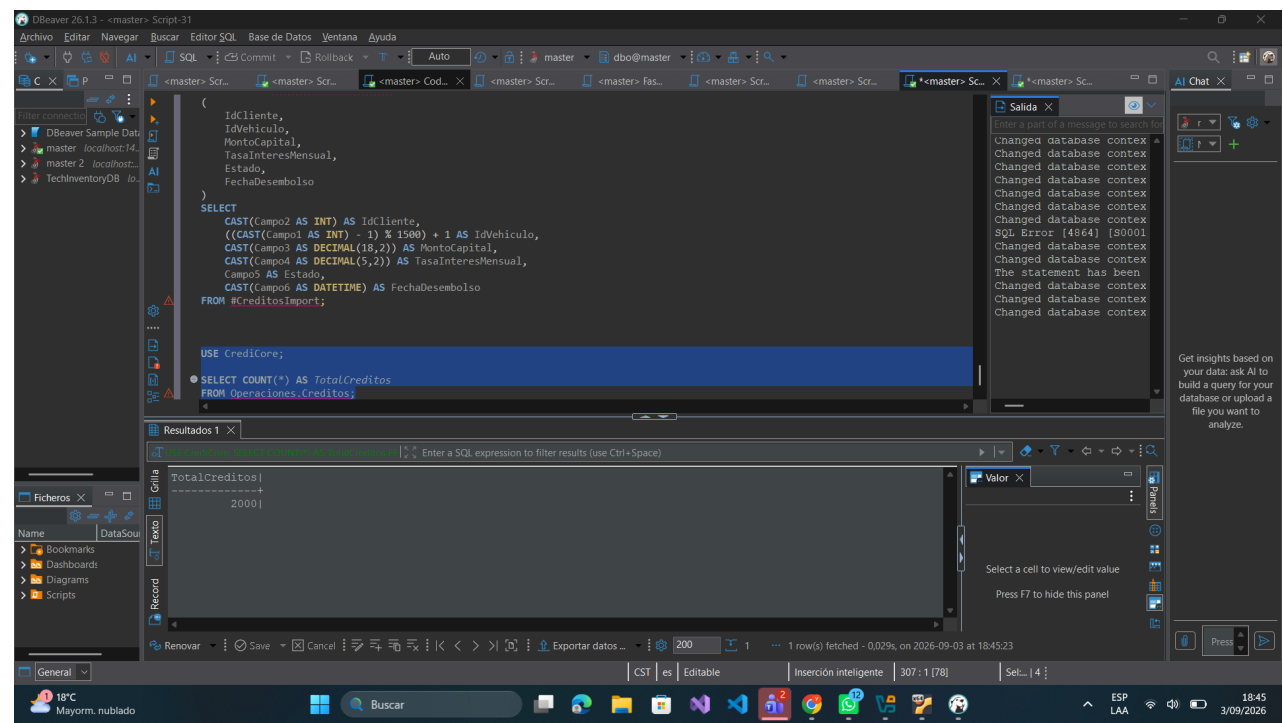
INSERT INTO Operaciones.Creditos ( IdCliente, IdVehiculo, MontoCapital, TasaInteresMensual, Estado,
FechaDesembolso ) SELECT CAST(Campo2 AS INT), ((CAST(Campo1 AS INT) - 1) % 1500) + 1, CAST(Campo3 AS
DECIMAL(18,2)), CAST(Campo4 AS DECIMAL(5,2)), Campo5, CAST(Campo6 AS DATETIME) FROM #CreditoImport;

```



Evidencia 11. Transformación previa a la inserción.

10. Verificación de 2,000 créditos



Evidencia 12. COUNT(*) confirma 2,000 créditos.

11. Investigación técnica — BULK INSERT

BULK INSERT es una instrucción T-SQL para importar grandes cantidades de información desde archivos hacia SQL Server. Frente a miles de INSERT individuales, permite procesar el archivo como una carga masiva y reducir el trabajo repetitivo de enviar y procesar una sentencia por fila. La mejora puede ser sustancial, aunque la velocidad final depende de almacenamiento, red, formato y configuración.

En este proyecto se utilizaron FIELDTERMINATOR='|' para separar columnas y ROWTERMINATOR='0x0a' para identificar el final de cada registro. Como SQL Server estaba dentro de Docker, el archivo tuvo que quedar visible dentro del contenedor y con permisos de lectura.

Un caso bancario sería la incorporación diaria de archivos de cartera o transacciones provenientes de sistemas externos para procesos ETL y análisis de riesgo. En telecomunicaciones puede utilizarse para cargar grandes archivos de registros de llamadas, consumo o eventos de red.

12. Parte B.1 — Riesgo acumulado

SELECT Estado, SUM(MontoCapital) AS TotalCapitalPrestado, AVG(TasaInteresMensual) AS PromedioTasaInteres FROM Operaciones.Credito GROUP BY Estado ORDER BY Estado;

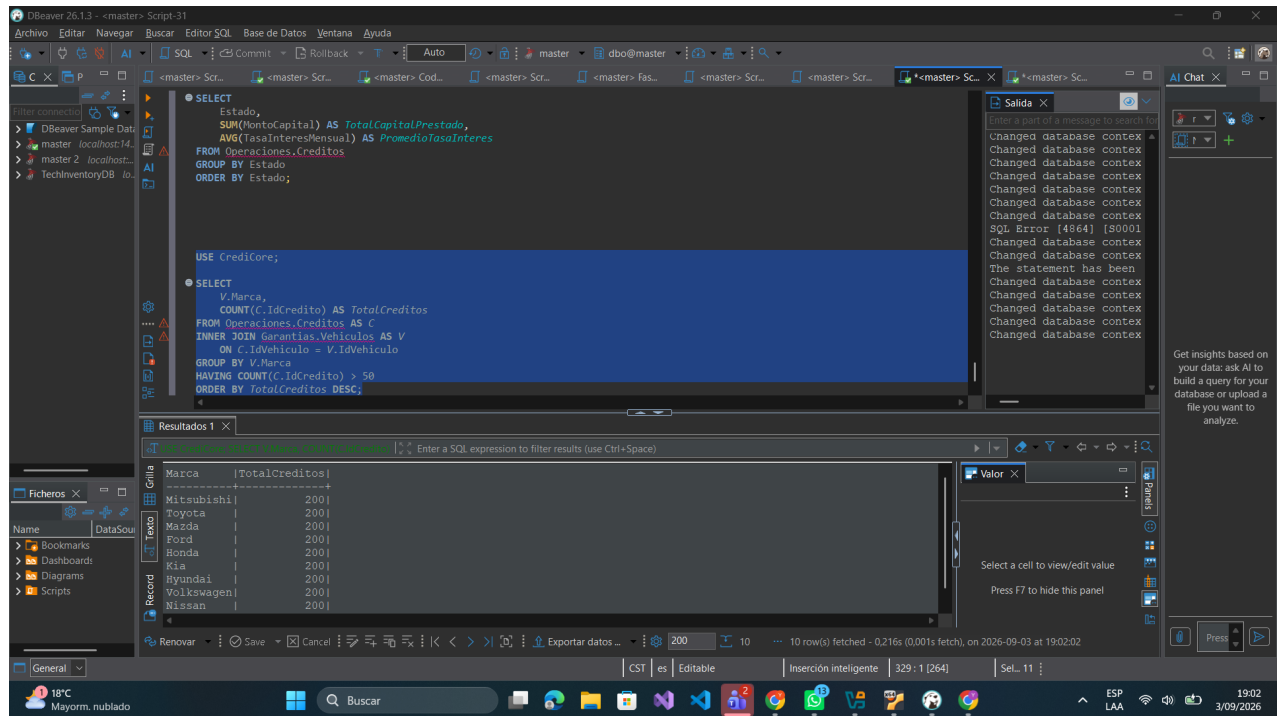
Estado	Total Capital	Promedio Tasa
Activo	Q7,619,340.00	3.500600
Cancelado	Q7,824,840.00	3.494060
Pagado	Q7,687,840.00	3.498420
Vencido	Q7,756,340.00	3.496240

Tiempo registrado en DBeaver: 0.373 segundos.

13. Parte B.2 — Concentración vehicular

```
SELECT V.Marca, COUNT(C.IdCredito) AS TotalCredito FROM Operaciones.Credito AS C INNER JOIN
Garantias.Vehiculos AS V ON C.IdVehiculo = V.IdVehiculo GROUP BY V.Marca HAVING COUNT(C.IdCredito) > 50 ORDER BY
TotalCredito DESC;
```

El resultado mostró diez marcas con 200 créditos cada una. Todas superan el umbral de 50 establecido mediante HAVING.



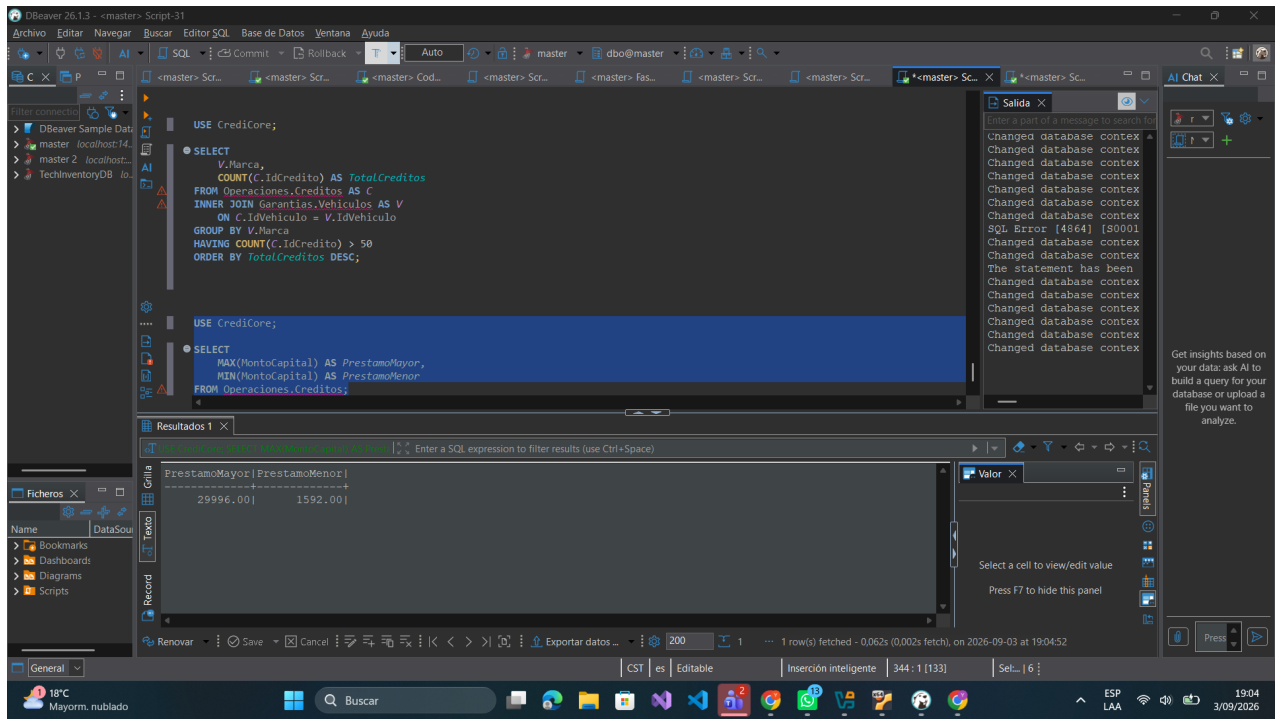
Evidencia 13. COUNT por Marca con HAVING > 50.

Tiempo registrado en DBeaver: 0.216 segundos. WHERE filtra filas antes del GROUP BY; HAVING filtra grupos después del GROUP BY. Aquí se usa HAVING porque la condición depende del COUNT de cada marca.

14. Parte B.3 — Análisis de extremos

```
SELECT MAX(MontoCapital) AS PrestamoMayor, MIN(MontoCapital) AS PrestamoMenor FROM Operaciones.Credito;
```

Préstamo mayor: Q29,996.00. Préstamo menor: Q1,592.00. Tiempo registrado en DBeaver: 0.062 segundos.



Evidencia 14. Resultado de MAX y MIN.

15. Resumen de resultados

Reporte	Funciones	Resultado	Tiempo
Riesgo acumulado	SUM + AVG + GROUP BY	4 estados	0.373 s
Concentración	COUNT + GROUP BY + HAVING	10 marcas; 200 c/u	0.216 s
Extremos	MAX + MIN	Q29,996 / Q1,592	0.062 s

16. Cumplimiento de la rúbrica

Rubro	Estado
A. GitHub — repositorio y commit solicitado	Pendiente de publicación
B. PDF técnico	Documentado en este reporte
C. Video YouTube 3–7 min	Pendiente de grabación

GitHub: _____

YouTube: _____

17. Conclusiones

La Fase 2 permitió completar un flujo de ingesta masiva y análisis sobre SQL Server 2022. Se demostró el límite de 1,000 filas y su solución mediante batches con GO; se automatizaron 500 INSERT con Excel; y se cargaron 2,000 créditos mediante archivo plano, staging y transformación.

La combinación de VirtualBox, Ubuntu, Docker y DBeaver permitió resolver el acceso al archivo y completar la carga. Finalmente, las tres consultas de Inteligencia de Negocios fueron ejecutadas exitosamente y sus resultados y tiempos quedaron documentados.

18. Archivos de soporte

- CrediCore_Fase2_Vehiculos_ERROR_1500.sql
- CrediCore_Fase2_Vehiculos_CORREGIDO_1500.sql
- CrediCore_Fase2_500_Clientes_Excel.xlsx
- creditos_2000.txt
- Scripts SQL de BULK INSERT, transformación y reportes.